

Documento de Diseño y Análisis

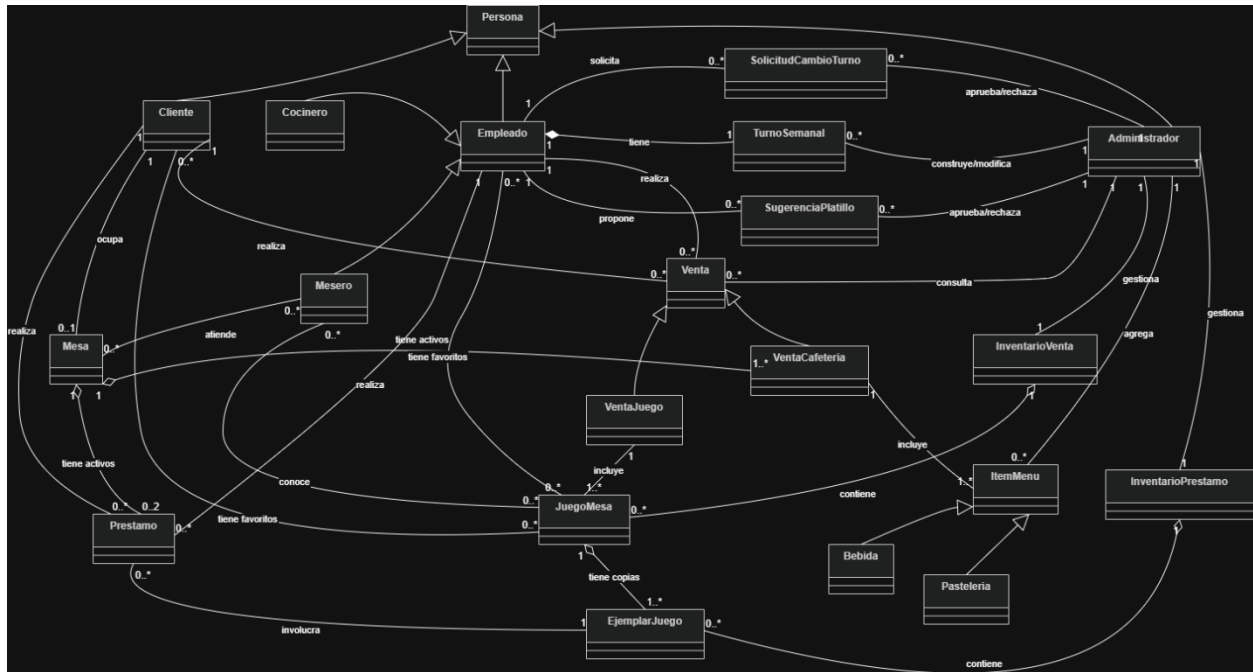
Nombres:

- Santiago Tarazona Upegui - 202513771
- Emilio Robledo
- Juan David Gómez Melo- 202510127

Se presenta el siguiente documento para mostrar el proceso de análisis y de diseño del proyecto relacionado al caso de Board Game Cafe, en este documento se tuvieron en cuenta las recomendaciones realizadas en la entrega uno, respecto a la organización de las clases en los distintos diagramas, para esto primero se aclara que cada diagrama tiene su respectivo png subido en el repositorio del proyecto en GitHub (\Entrega2\diagramas), no se realizaron mayores cambios a la lógica del programa planteada en la primera entrega, solo se cambiaron los diagramas para mayor claridad según lo recomendado.

Modelo de Dominio:

Diagrama de clases:



Entidades del dominio:

Persona (abstracta)

- nombre
- apellido
- correoElectrónico
- contraseña

- login

Cliente (*extiende Persona*)

- puntosFidelidadAcumulados

Empleado (*abstracta, extiende Persona*)

Mesero (*extiende Empleado*)

Cocinero (*extiende Empleado*)

Administrador (*extiende Persona*)

JuegoMesa

- nombre
- añoPublicación
- empresaFabricante
- categoría (Cartas / Tablero / Acción)
- minimoJugadores
- maximoJugadores
- restriccionEdad (apto menores de 5 años / exclusivo adultos / sin restricción)
- esDifícil
- precioDeVenta

EjemplarJuego

- estado (Nuevo / Bueno / Falta una pieza / Desaparecido / etc.)
- numeroDeVecesPrestado
- disponible

InventarioPrestamo

- capacidad máxima

InventarioVenta

- capacidad máxima

Mesa

- numeroMesa

- maximaDePersonas

Prestamo

- fechaHoraInicio
- fechaHoraDevolución
- estado (activo / devuelto / desaparecido)

ItemMenu (*abstracta*)

- nombre
- descripcion
- precioBase

Bebida (*extiende ItemMenu*)

- esAlcohólica
- temperatura (caliente / fría)

Pasteleria (*extiende ItemMenu*)

- listaAlérgenos (maní, mariscos, gluten, etc.)

SugerenciaPlatillo

- descripción del platillo sugerido
- estado (pendiente / aprobada / rechazada)
- fechaSugerencia

Venta (*abstracta*)

- fechaHora
- subtotal
- totalConImpuestos
- descuentoAplicado
- puntosFidelidadGenerados

VentaJuego (*extiende Venta*)

- IVA (19%)

VentaCafeteria (*extiende Venta*)

- IMPUESTOCONSUMO (8%)

- propina

TurnoSemanal

- díaSemana
- horaInicio
- horaFin

SolicitudCambioTurno

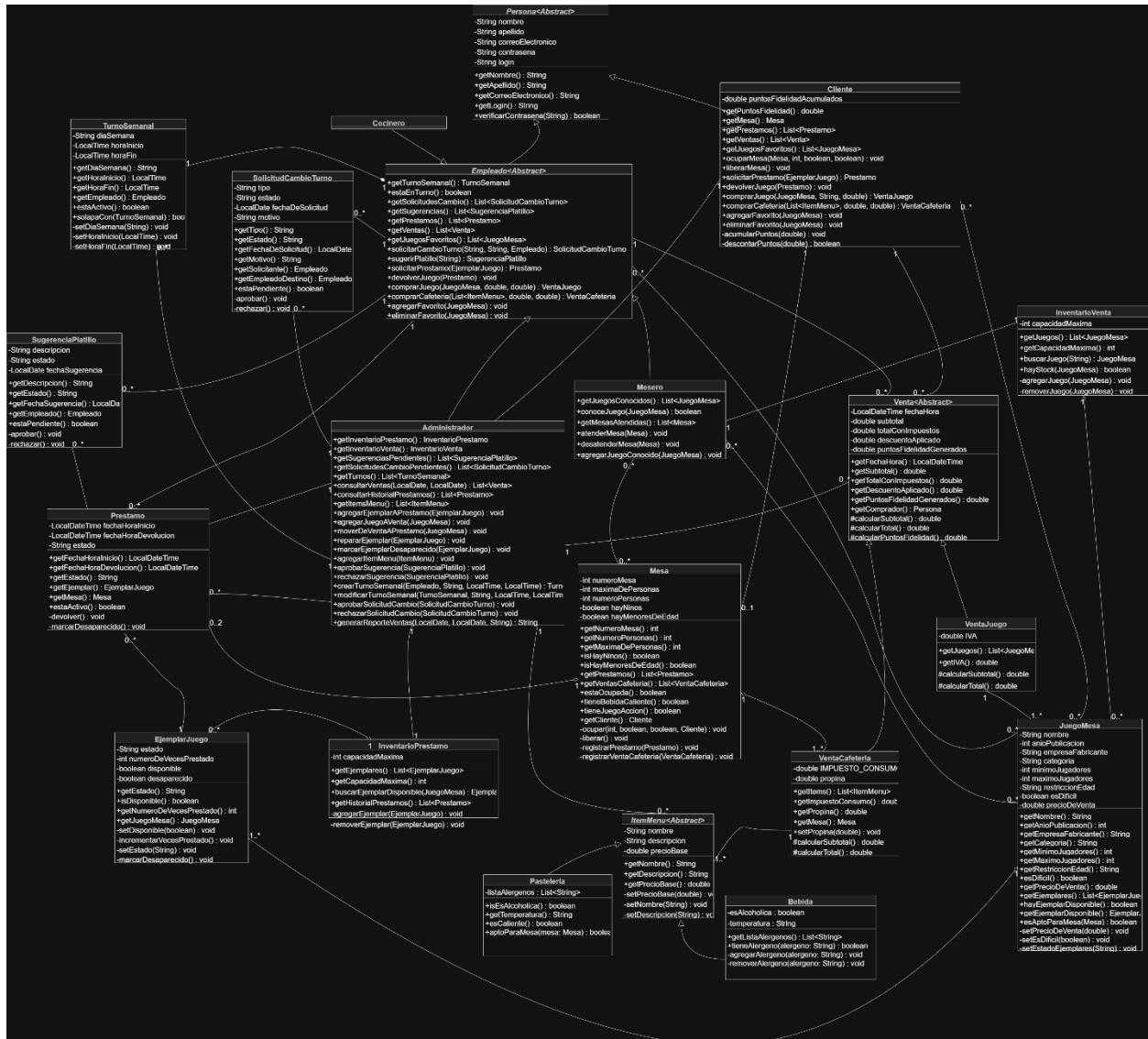
- tipo (cambio general / intercambio con otro empleado)
- estado (pendiente / aprobada / rechazada)
- fechaDeSolicitud
- motivo

Relaciones del modelo del dominio

- **Persona** es generalización de **Cliente** (1 a 1)
- **Persona** es generalización de **Empleado** (1 a 1)
- **Persona** es generalización de **Administrador** (1 a 1)
- **Empleado** es generalización de **Mesero** (1 a 1)
- **Empleado** es generalización de **Cocinero** (1 a 1)
- **ItemMenu** es generalización de **Bebida** (1 a 1)
- **ItemMenu** es generalización de **Pasteleria** (1 a 1)
- **Venta** es generalización de **VentaJuego** (1 a 1)
- **Venta** es generalización de **VentaCafeteria** (1 a 1)
- **JuegoMesa** tiene copias de **EjemplarJuego** (1 a 1..*)
- **InventarioPrestamo** contiene **EjemplarJuego** (1 a 0..*)
- **InventarioVenta** contiene **JuegoMesa** (1 a 0..*)
- **Mesa** tiene activos **Prestamo** (1 a 0..2)
- **Mesa** tiene activos **VentaCafeteria** (1 a 1..*)
- **Prestamo** involucra **EjemplarJuego** (0..* a 1)
- **VentaJuego** incluye **JuegoMesa** (1 a 1..*)
- **VentaCafeteria** incluye **ItemMenu** (1 a 1..*)
- **Cliente** ocupa **Mesa** (1 a 0..1)
- **Cliente** realiza **Prestamo** (1 a 0..*)
- **Cliente** realiza **Venta** (1 a 0..*)
- **Cliente** tiene favoritos **JuegoMesa** (0..* a 0..*)
- **Mesero** conoce (juegos difíciles) **JuegoMesa** (0..* a 0..*)
- **Mesero** atiende **Mesa** (0..* a 0..*)
- **Empleado** tiene **TurnoSemanal** (1 a 1)
- **Empleado** solicita **SolicitudCambioTurno** (1 a 0..*)

- **Empleado** propone **SugerenciaPlatillo** (1 a 0..*)
- **Empleado** realiza **Prestamo** (1 a 0..*)
- **Empleado** realiza (como cliente) **Venta** (1 a 0..*)
- **Empleado** tiene favoritos **JuegoMesa** (0..* a 0..*)
- **Administrador** gestiona **InventarioPrestamo** (1 a 1)
- **Administrador** gestiona **InventarioVenta** (1 a 1)
- **Administrador** aprueba/rechaza **SugerenciaPlatillo** (1 a 0..*)
- **Administrador** aprueba/rechaza **SolicitudCambioTurno** (1 a 0..*)
- **Administrador** construye/modifica **TurnoSemanal** (1 a 0..*)
- **Administrador** consulta **Venta** (1 a 0..*)
- **Administrador** agrega **ItemMenu** (1 a 0..*)

Diagrama de alto nivel:



DECISIONES DE DISEÑO

El diseño del sistema se realizó con reutilización y separación de responsabilidades, con el objetivo de construir una solución clara, extensible y mantenible.

Uso de herencia y abstracción

Se utilizó herencia para modelar los diferentes tipos de usuarios del sistema a partir de la clase abstracta Persona, esto permitió centralizar atributos comunes como nombre, login y contraseña, evitando duplicación de código. Adicionalmente, la clase Empleado se definió como abstracta, permitiendo especializaciones como Mesero y Cocinero, cada una con comportamientos específicos. También se utilizaron clases abstractas como Venta e ItemMenu para representar conceptos generales que luego se especializan en clases concretas como VentaJuego, VentaCafeteria, Bebida y Pasteleria.

Separación entre modelo y persistencia

El sistema se dividió en dos capas principales:

- Modelo: contiene la lógica del negocio (usuarios, ventas, préstamos, mesas, etc.).
- Persistencia: se encarga del almacenamiento y recuperación de los datos mediante archivos.

Adicionalmente, se implementó un manejo de excepciones para controlar errores en tiempo de ejecución, como problemas de disponibilidad, capacidad o acceso a datos.

Esta separación permite:

- Facilitar cambios futuros en la forma de almacenamiento
- Mantener el código organizado
- Mejorar la mantenibilidad del sistema
- Asegurar un manejo adecuado de errores

Manejo de relaciones

Las relaciones entre objetos se implementaron mediante listas (List), lo que permite modelar relaciones de uno a muchos de manera flexible.

Por ejemplo:

- Un cliente puede tener múltiples préstamos y ventas

- Un inventario puede contener múltiples juegos o ejemplares
- Un mesero puede atender múltiples mesas

Encapsulamiento

Se implementó encapsulamiento mediante atributos privados y métodos públicos para proteger la integridad de los datos.

Además, varias clases manejan estados internos, como:

- SolicitudCambioTurno (pendiente, aprobada, rechazada)
- Prestamo (activo, devuelto, desaparecido)

Validaciones del dominio

Se implementaron validaciones directamente en las clases del modelo, tales como:

- Restricciones de edad en juegos
- Número de jugadores permitidos
- Disponibilidad de ejemplares
- Capacidad máxima de inventarios

Esto garantiza que las reglas del negocio se cumplan en todo momento.

Uso del polimorfismo

El sistema hace uso de polimorfismo al manejar listas de tipos generales como Persona o Venta, que pueden contener instancias de sus subclases. Esto permite mayor flexibilidad y simplifica la lógica del sistema.

Diseño orientado al dominio del problema

El modelo fue diseñado con base en el funcionamiento real de un Board Game Café, incluyendo:

- Gestión de mesas
- Préstamo de juegos
- Ventas de productos
- Turnos de empleados

Esto permite que el sistema represente fielmente el contexto del problema.

Posibles mejoras

Como mejoras futuras del sistema se podrían implementar:

- Uso de una base de datos en lugar de archivos
- Interfaz gráfica para mejorar la experiencia del usuario
- Implementación de patrones de diseño
- Validaciones más avanzadas en reglas de negocio

Asignación de responsabilidades

A continuación, se describen las responsabilidades principales de cada componente del sistema, con el fin de garantizar una correcta distribución de funciones y evitar sobrecarga en las clases.

Persona

- Representar un usuario del sistema
- Gestionar información básica (nombre, login, contraseña)
- Permitir autenticación

Cliente

- Ocupar y liberar mesas
- Solicitar y devolver préstamos de juegos
- Realizar compras (juegos y cafetería)
- Gestionar puntos de fidelidad
- Administrar lista de juegos favoritos

Empleado

- Representar comportamiento común de empleados
- Gestionar solicitudes de cambio de turno
- Realizar préstamos y ventas
- Gestionar juegos favoritos

Mesero

- Atender mesas
- Gestionar mesas asignadas

- Conocer y recomendar juegos
- Enseñar juegos difíciles

Cocinero

- Gestionar sugerencias de platillos
- Participar en el flujo de pedidos de cafetería

Administrador

- Gestionar inventarios de préstamo y venta
- Aprobar o rechazar solicitudes de cambio de turno
- Aprobar o rechazar sugerencias de platillos
- Gestionar turnos semanales
- Consultar ventas e historial de préstamos
- Generar reportes

JuegoMesa

- Representar un juego de mesa
- Validar restricciones de uso (edad, jugadores)
- Gestionar sus ejemplares

EjemplarJuego

- Representar una copia física de un juego
- Indicar disponibilidad
- Llevar control de uso y estado

InventarioPréstamo

- Gestionar ejemplares disponibles para préstamo
- Controlar capacidad máxima
- Buscar ejemplares disponibles

InventarioVenta

- Gestionar juegos disponibles para venta
- Controlar capacidad máxima

Mesa

- Representar una mesa del establecimiento

- Gestionar clientes asociados
- Registrar préstamos y ventas de cafetería
- Validar restricciones (bebidas, juegos, etc.)

Prestamo

- Registrar préstamos de juegos
- Controlar estado del préstamo
- Gestionar devolución o pérdida

Venta

- Representar una transacción
- Calcular subtotal y total

VentaJuego

- Gestionar venta de juegos
- Aplicar IVA

VentaCafeteria

- Gestionar pedidos de cafetería
- Aplicar impuesto al consumo
- Gestionar propina

ItemMenu

- Representar un producto del menú

Bebida

- Representar bebidas
- Gestionar temperatura y contenido alcohólico

Pasteleria

- Representar productos de pastelería
- Gestionar alérgenos

TurnoSemanal

- Representar horarios de empleados
- Validar solapamiento de turnos

SolicitudCambioTurno

- Representar solicitudes de cambio de turno
- Gestionar estado de la solicitud

SugerenciaPlatillo

- Representar sugerencias de nuevos productos
- Gestionar estado de aprobación

Requerimientos funcionales:

Cliente:

Registro e inicio de sesión

El cliente debe poder iniciar sesión en el sistema utilizando un login y una contraseña.

Consultar catálogo de juegos

El cliente puede consultar el catálogo de juegos disponibles en el café, incluyendo características como:

- categoría
- número de jugadores
- edad mínima
- dificultad

Reservar mesa

El cliente debe poder ocupar una mesa indicando:

- número de personas
- si hay niños
- si hay menores de edad.

Solicitar préstamo de juegos

El cliente puede solicitar el préstamo de hasta dos juegos simultáneamente para su mesa.

El sistema debe verificar:

- disponibilidad del juego
- número de jugadores
- restricciones de edad.

Devolver juegos

El cliente debe poder devolver los juegos prestados al finalizar su estadía.

Comprar juegos de mesa

El cliente puede comprar juegos disponibles en el inventario de ventas.

El sistema debe:

- registrar la compra
- aplicar IVA (19%)
- registrar la venta para contabilidad.

Comprar productos de cafetería

El cliente puede comprar productos del menú (bebidas y pastelería).

El sistema debe:

- aplicar impuesto al consumo (8%)
- permitir incluir propina.

Recibir advertencias de alérgenos

El sistema debe advertir al cliente si un producto contiene alérgenos.

Guardar juegos favoritos

El cliente puede guardar juegos como favoritos.

Acumular puntos de fidelidad

El cliente recibe 1% del valor de la compra en puntos de fidelidad, que pueden usarse como descuento en compras futuras.

Empleado:**Iniciar sesión en el sistema**

Todos los empleados deben autenticarse con usuario y contraseña.

Consultar turnos semanales

El empleado puede consultar su horario de trabajo semanal.

Solicitar cambio de turno

El empleado puede solicitar:

- cambio de turno

- intercambio con otro empleado.

Estas solicitudes deben ser aprobadas por el administrador.

Sugerir nuevos platillos

Los empleados pueden sugerir nuevos productos para el menú del café.

Comprar juegos con descuento

Los empleados pueden comprar juegos con un descuento del 20%.

También pueden compartir un código de 10% de descuento con clientes.

Alquilar juegos

Los empleados pueden alquilar juegos únicamente cuando:

- no estén en turno laboral
- no haya clientes que atender.

Enseñar juegos difíciles

Los meseros pueden enseñar juegos catalogados como difíciles si están capacitados para hacerlo.

Administrador:

Gestionar inventario de juegos

El administrador puede:

- agregar juegos al inventario
- reabastecer inventario
- mover juegos entre inventario de venta y préstamo.

Reparar juegos

El administrador puede reemplazar un juego dañado usando una copia del inventario de ventas.

Marcar juegos como desaparecidos

Si un juego no se devuelve después de varios días, el administrador puede marcarlo como desaparecido.

Consultar historial de préstamos

El administrador puede ver:

- quién prestó un juego
- cuando se prestó
- cuántas veces ha sido prestado
- estado del juego.

Gestionar el menú

El administrador puede:

- aprobar sugerencias de platillos
- agregar nuevos productos al menú.

Gestionar turnos

El administrador puede:

- crear turnos semanales
- modificar turnos
- aprobar o rechazar solicitudes de cambio.

Consultar reportes financieros

El administrador puede consultar reportes de ventas:

- diarias
- semanales
- mensuales.

Estos reportes deben incluir:

- ingresos
- impuestos
- propinas
- ventas de juegos
- ventas de comida.

Restricciones del proyecto

Persistencia de datos:

Toda la información del sistema debe almacenarse en archivos para garantizar que los datos permanezcan entre ejecuciones.

Autenticación de usuarios:

Todos los usuarios del sistema deben acceder mediante un login y una contraseña.

Capacidad del establecimiento:

El sistema debe respetar la capacidad máxima del café, rechazando nuevas reservas si se supera este límite.

Préstamo de juegos:

Cada mesa puede tener máximo dos juegos prestados simultáneamente, siempre que haya disponibilidad y se cumplan las restricciones de edad y número de jugadores.

Restricciones del menú:

- No se pueden servir bebidas alcohólicas a mesas con menores de edad.
- No se pueden servir bebidas calientes en mesas con juegos de categoría Acción.

Turnos de empleados:

El café debe tener al menos 1 cocinero y 2 meseros en turno simultáneamente, por lo que los cambios de turno no pueden violar esta condición.

Programas de prueba:

Dado que en esta entrega no se requiere una interfaz gráfica, se desarrollarán programas de prueba en consola para demostrar que las funcionalidades del sistema funcionan correctamente.

Los programas de prueba demostrarán:

Programa 1: Gestión de juegos

- Cargar catálogo de juegos
- Consultar juegos disponibles
- Ver información de un juego

Programa 2: Préstamo de juegos

- Crear una mesa
- Solicitar préstamo de juegos
- Validar restricciones de edad y número de jugadores
- Devolver juegos

Programa 3: Ventas de juegos

- Registrar compra de juegos
- Aplicar IVA
- Registrar puntos de fidelidad

Programa 4: Ventas de cafetería

- Registrar pedido de bebidas y pastelería
- Calcular impuesto al consumo
- Calcular propina

Programa 5: Gestión de turnos

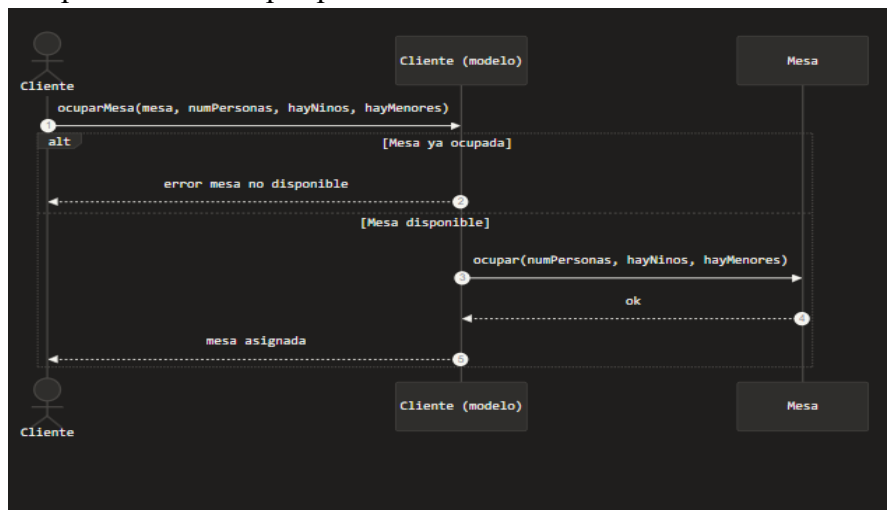
- Consultar turnos de empleados
- Solicitar cambio de turno
- Aprobar o rechazar solicitudes

Programa 6: Administración

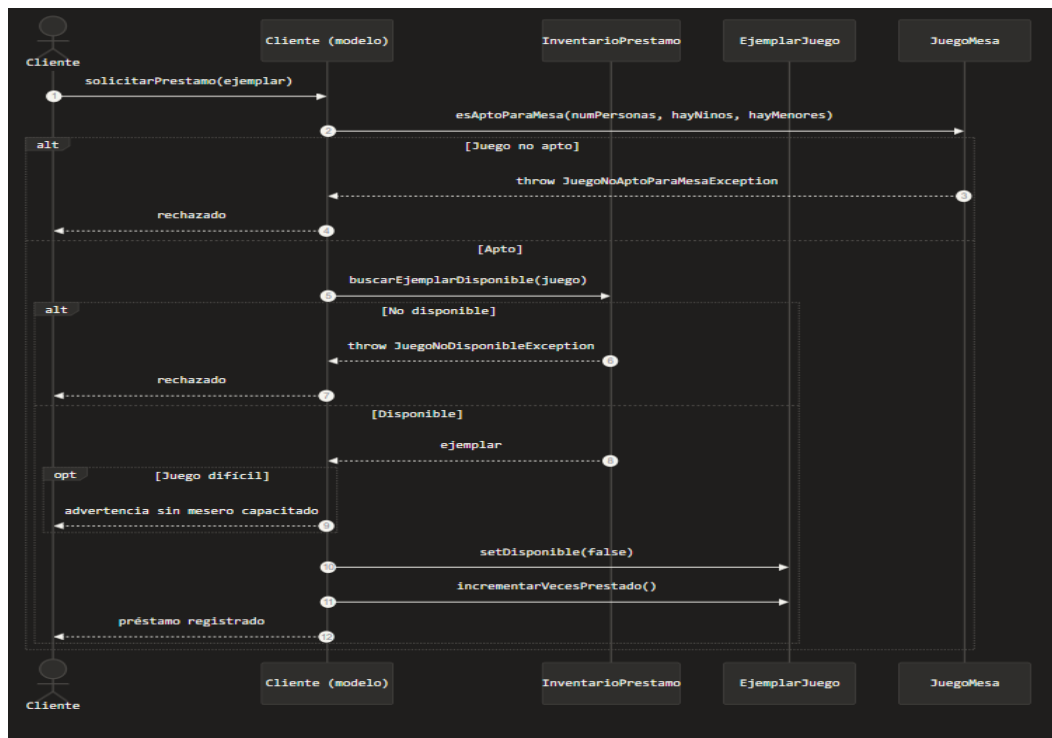
- Consultar inventario de juegos
- Registrar historial de préstamos
- Generar reporte de ventas

Diagramas de secuencia para las funcionalidades principales del inicio del sistema:

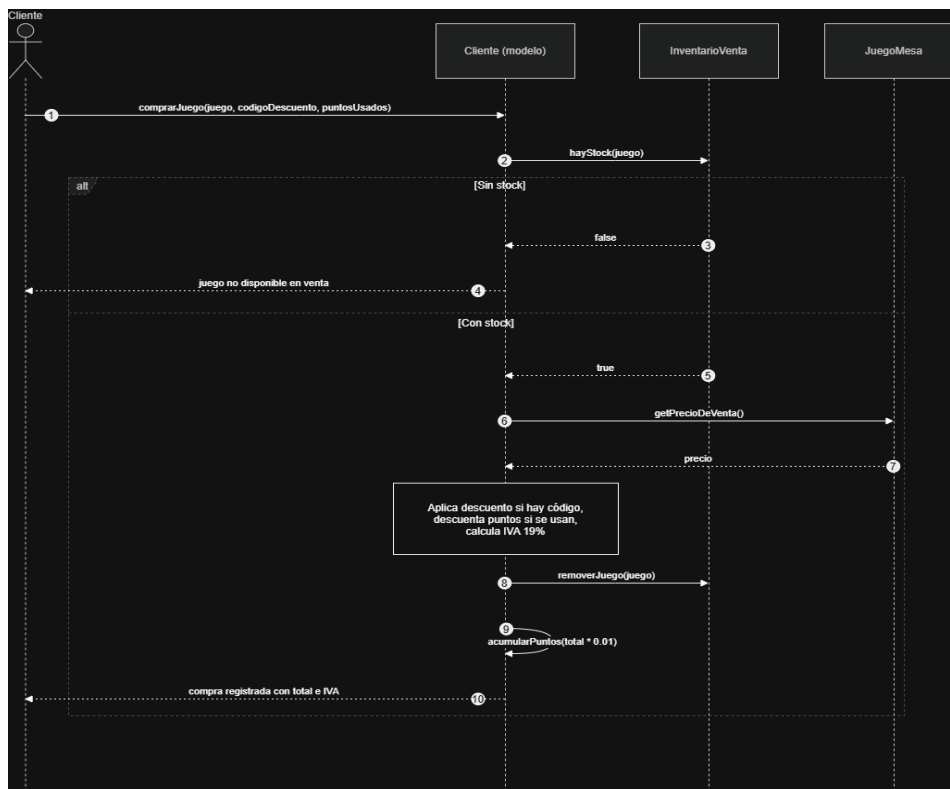
Ocupación de mesa por parte de cliente:



Solicitud de préstamo por parte del cliente:



Comprar Juego por parte del cliente:



```
sequenceDiagram
    participant Cliente as Cliente (modelo)
    participant Mesa
    participant Ubidia
    participant Pasteria

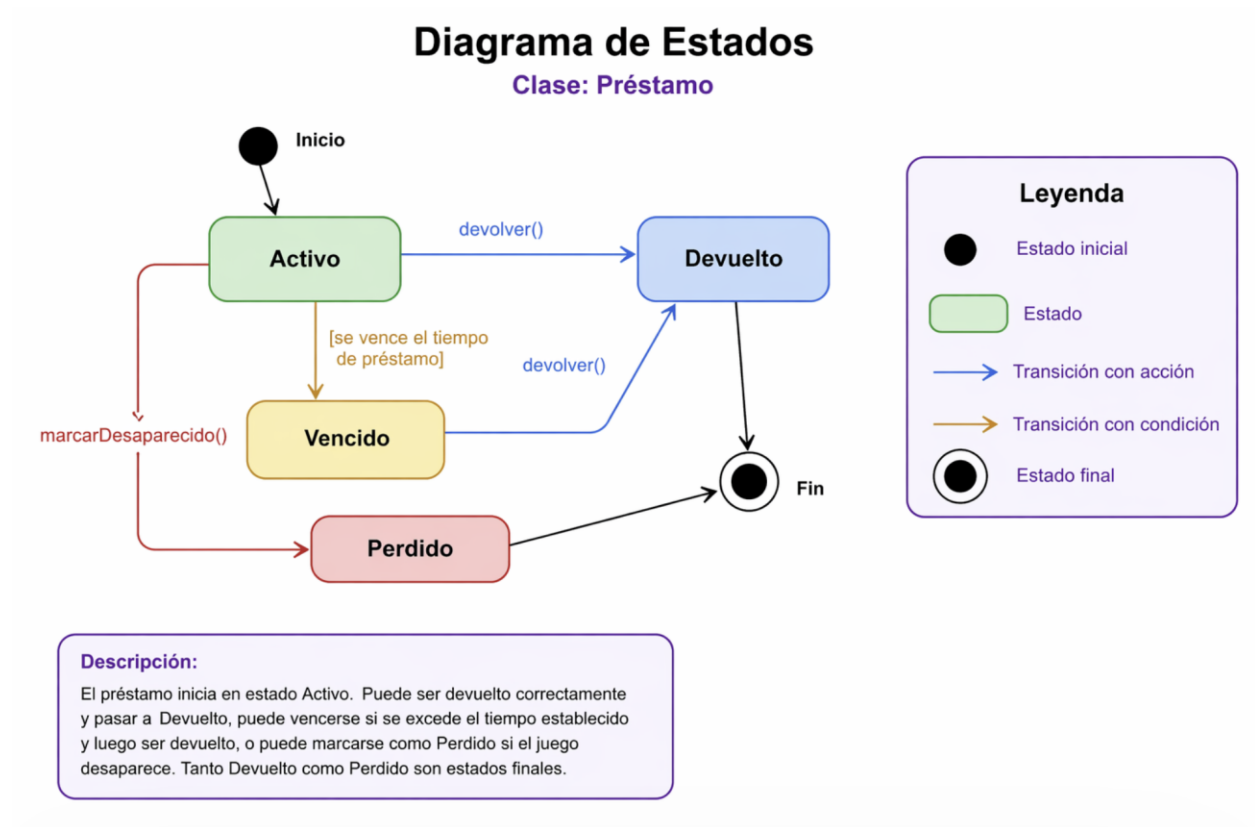
    Cliente->>Mesa: compareCafes(tarjetas, propina, puntosRebados)
    activate Mesa
    Mesa->>Ubidia: [Por cada item]
    activate Ubidia
    Ubidia->>Pasteria: [Eli bebida]
    activate Pasteria
    Pasteria->>Ubidia: getInstanciaBebida()
    deactivate Pasteria
    Ubidia->>Mesa: [Alcohólica con menores]
    activate Mesa
    Mesa->>Ubidia: throw BadRequestException
    deactivate Mesa
    Ubidia->>Ubidia: [Cambio con pago de Accion]
    activate Ubidia
    Ubidia->>Pasteria: throw BadRequestException
    deactivate Ubidia
    Ubidia->>Mesa: [Paga]
    activate Mesa
    Mesa->>Ubidia: [Fin Pasteria]
    deactivate Mesa
    Ubidia->>Pasteria: getInstanciaAlimento()
    activate Pasteria
    Pasteria->>Ubidia: alimento
    deactivate Pasteria
    Ubidia->>Ubidia: [Hay alergenico]
    activate Ubidia
    Ubidia->>Ubidia: advertencia de alergenico
    deactivate Ubidia
    Ubidia->>Ubidia: Calcula subtotal = propina + impuesto consumo 0%
    activate Ubidia
    Ubidia->>Ubidia: subTotal = (subTotal * 0.01)
    deactivate Ubidia
    Ubidia->>Mesa: venta registrada
    deactivate Ubidia
    Mesa->>Ubidia: [Fin Ubidia]
    deactivate Mesa
    Ubidia->>Ubidia: [Fin Ubidia]
    deactivate Ubidia
```

```
sequenceDiagram
    actor Usuario
    participant Empleado as Empleado (modelo)
    participant Solicitud as SolicitudCambioTurno

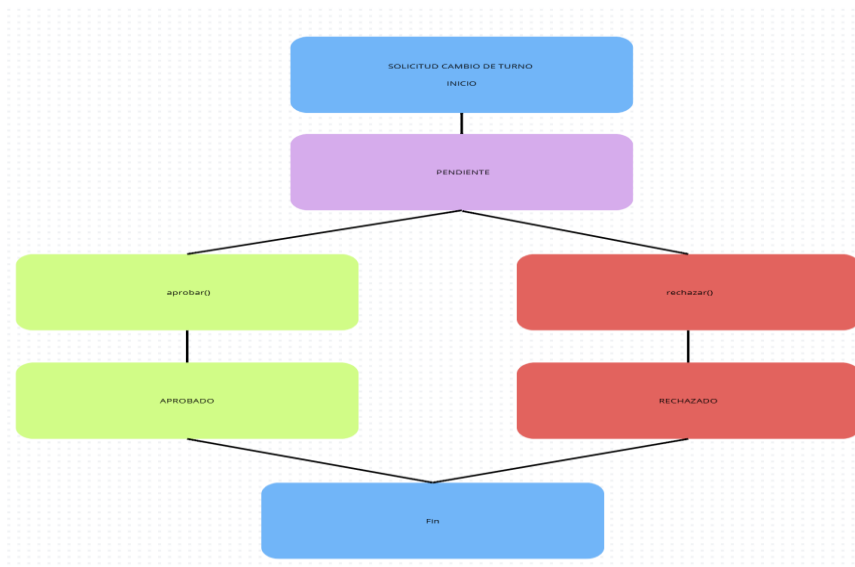
    Usuario->>Empleado: 1 solicitarCambioTurno(tipo, motivo, otroEmpleado)
    alt [tipo = "cambio"]
        Empleado->>Solicitud: 2 new SolicitudCambioTurno("cambio", motivo, empleado, turno, null, null)
        Solicitud-->>Empleado: 3 solicitud en estado pendiente
        Empleado-->>Usuario: 4 solicitud enviada al administrador
    else [tipo = "intercambio"]
        Empleado->>Solicitud: 5 new SolicitudCambioTurno("intercambio", motivo, empleado, turno, otroEmpleado, turnoOtro)
        Solicitud-->>Empleado: 6 solicitud en estado pendiente
        Empleado-->>Usuario: 7 solicitud de intercambio enviada al administrador
    end
    alt [tipo = "otro"]
        Empleado-->>Usuario: 8 solicitud de otro turno enviada al administrador
    end
    alt [tipo = "otro"]
        Empleado-->>Solicitud: 9 new SolicitudCambioTurno("otro", motivo, empleado, turno, otroEmpleado, turnoOtro)
        Solicitud-->>Empleado: 10 solicitud en estado pendiente
        Empleado-->>Usuario: 11 solicitud de otro turno enviada al administrador
    end
```

Diagrama de Estados:

Clase Préstamo:



Clase SolicitudCambioDeTurno:



Clase EjemplarJuego:

